

Bar Inventory & Operations Database

DB Design Write-Up — Schema: YLarkin

Yvonne Larkin

Spring 2026

Scenario

This database was designed for a bar or small restaurant to manage its inventory for products used in the bar. The products are featured in cocktail menu, vendor relationships, employee records, and financial transactions.

The establishment purchases products from vendors via **Purchase Orders** and sells cocktails to guests via **Sales Receipts**. Both transaction types feed into a shared **TXN_Transaction** supertype, allowing unified bookkeeping across inbound and outbound cash flow. The cocktail menu is built from an internal product inventory that tracks individual spirits, wines, and brews by category, subtype, and volume — both front-of-house (FOH bar) and back-of-house (BOH storage). Employees are linked to sales receipts to track server or bartender activity.

In practice, a typical workflow runs as follows: a **vendor** ships product on an invoice, which is recorded as a Purchase Order with one or more line items. Those products enter **inventory**, where their volumes are tracked. When a guest orders a cocktail, the bartender (an **employee**) rings it as a Sales Receipt line item, drawing from the cocktail's stored recipe. The recipe itself maps cocktail IDs back to specific product IDs, providing traceability from guest order all the way to the bottle on the shelf.

Model Walkthrough

The Central Entity and Event

The central entity to this model is the **product_id** it is the crux of multiple tables to directly refer to. The Inventory Table generates the PK for the entity and that PK is used as the PK of multiple parent to child 1:1 relationships. This entity PK serves as a major identity layer that all category-specific tables (Liquor, Brew, Wine) reference. Both the 'Cocktail Recipe' table and the 'Purchase Order' line item table point back to **product_id** in this important table to ensure product identity across multiple transaction.

Which brings me to the Central Event, **TXN_Transaction_14**, similar to the *product_id* the table event generates a **transaction_id** PK that is used directly as the PK in the 2 major Child Events distinguished by **subtype** ENUM values of 'Purchase Order' or 'Sales Receipt'. This ***transaction_id*** subsequently further used in a 1:many relationship for the children to PO and SR (TXNsub_PO_15 and TXNsub_SR_17) where a composite key is used to make distinct values in the table.

Both are Central to the DB, but the purpose of the DB use would determine which table is more important for the DB, but both of these Central Entities create multi-use PK identities.

This design was intentionally prevented from further normalization in the interest of time. There are many ways to build and deepen the model use in business. I kept large supertype to subtype normalization base, and tried to reduce the expansion too far without everything being Type 1 Normalization.

The Three Specializations

For the purpose of this project, I will focus on the Brew, Wine, and Liquor entities. There are other examples throughout of poorer normalization in accessory tables and some degree of parent/child entities as well, but I won't highlight those here.

As a note: I did make one over-arching alcohol table (the inventory table mentioned above) this was so that a unique id was guaranteed across all products to reduce confusion and aid in central index of the `product_id`.

Specialization Type I — Brew — Single Table with Nulls (No Subtypes)

This pattern is used for **Brew** (`BREW_10`), which covers Beer, Cider, Hard Seltzer, and Other (ENUM) categories in a single table.

The table uses the ENUM `category` to broadly group the table values but the entity still has multiply boolean flags — `is_NA`, `is_Hazy`, `is_nitro`, `is_craft`, `is_seasonal` — that apply meaningfully across all Beer types, but not 'Cider', 'Seltzer', or 'Other'. While not every column is relevant to 'Beer' products the values are not automatically ruled out like for the non-Beer counterparts making the table have NULLs by default. For example, `IBU` (International Bitterness Units) will be null for a hard seltzer but populated for an IPA; `hop_profile` will be null for a cider but present for a pale ale.

This specialization was favored here because the brew categories are closely related in attributes, the table stays manageable in width, and the number of nullable columns is small relative to the total column count. Splitting beers, ciders, and seltzers into three separate tables would create three nearly-identical schemas with no meaningful differentiation in the data they hold.

Specialization Type II — Wine — Subtype Tables Only (No Shared Supertype Table)

This pattern is used for **Wine** (`WINE_Red_11`, `WINE_Rose_11`, `WINE_White_11`).

Each wine color (subtype) is stored in its own separate table. All three tables have nearly identical column structures — varietal style, vintage year, ABV, appellation, terroir region, residual sugar class, gluten-free flags — but no shared 'Wine' supertype table exists. Each table references `inventory_product_start_6` directly via foreign key.

This design was chosen because the semantic differences between red, rosé, and white wine are significant from a product and operations standpoint. A bar manager filtering inventory by color, or querying specific varietals, benefits from naturally scoped queries against a single focused table. Adding a supertype that carries no additional differentiating data would only introduce an unnecessary join on every wine query.

Specialization Type III — Liquor — Supertype Table + Subtype Tables

The **Liquor** pattern (`LIQUOR_8` + `LIQsub*_9` tables):

Spirits share a common supertype (`LIQUOR_8`) that stores attributes universal to all liquor — ABV, bottle size, country of origin, and a `subtype_LUT` tag. Each spirit category then has its own subtype table carrying attributes specific only to that type:

- `LIQsub_Vodka_9` — filtration method, water source, distillation count
- `LIQsub_Tequila_9` — agave variety, cooking method, NOM number, additives profile
- `LIQsub_Gin_9` — botanical bill, maceration method, juniper intensity, number of botanicals
- `LIQsub_Rum_9` — base type, agricole flag, sugar added, pot/column ratio
- `LIQsub_Whiskey_9` — grain profile, peat level, cask finish, single malt / single barrel flags
- `LIQsub_liqueur_9` — liqueur family, sweetness level, sugar content, cream-based and amaro flags

This specialization was chosen because while all spirits share a category identity, each type stores a meaningfully different and non-overlapping set of attributes. A whiskey is the only liquor that uses peat level in PPM, but tequila uses NOM certification number and while there are *stronger* abv version the terminology varies (cask strength in whiskey vs navy strength in rum). Collapsing these into one table would produce a very wide,

very sparse table. Separate subtypes keep each table focused, eliminate excess null columns, and reflects the intuitive way people think about spirits.

Transactions (TXN_Transaction_14 + TXNsub_PO_15 / TXNsub_SR_17):

The transaction supertype unifies all financial events while the subtypes carry transaction-specific detail. A Purchase Order subtype needs invoice number, received date, and an `is_paid_in_full` flag. A Sales Receipt subtype needs tip amount, tendered amount, payment method, and a link to the serving employee. These data are distinct enough — and important enough for their respective workflows — to warrant separate subtype tables rather than a single wide transaction table with many nulls.

Cocktail Menu (MENU_Cocktail_start_12 + MENUsub_Cocktail_LineRecipe_13): The cocktail entity is paired with a recipe line item table. The parent table stores the cocktail identity, pricing tiers (single, double, happy hour), and flavor profile. The child table is the ingredient-level recipe: each row is one ingredient line within one cocktail, carrying the product ID, ingredient name, subtype classification (e.g., 'Liqueur', 'Non-Alcoholic', 'Garnish'), and the measure for single vs. double pours. This relationship is modeled as a supertype-subtype pair rather than embedding recipe data directly in the cocktail table, since a cocktail may have 1–10+ ingredient lines and the line-level data has a meaningfully different shape than the cocktail-level metadata.

Model Limitations

1. **No direct link between a cocktail sale and inventory depletion.** The Sales Receipt line item table records which cocktails were sold, and the recipe table records which products each cocktail requires, but no automatic or recorded deduction of volume occurs in `inventory_volumes_7` when a sale is made. A trigger or a dedicated depletion log table would be needed to support real-time perpetual inventory. I did not have the time to do this.
2. **Product ID in Purchase Order line items is nullable and partially denormalized.** `TXN_PO_line_item_16` stores `product_id` as nullable alongside `product_brand` and `category_type` as free-text fields, partially duplicating data already in `inventory_product_start_6`. This was a practical allowance for recording shipments of products not yet registered in inventory, but it creates a risk of inconsistency. There are other Data Type flaws that were unaddressed due to time as well.
3. **Wines share no supertype, requiring UNION for wine-wide queries.** Because the three wine color tables reference `inventory_product_start_6` directly with no shared Wine supertype, a query listing all wines in inventory requires a `UNION` across three tables. This is workable with a view, but any wine-wide report adds structural overhead that a supertype would eliminate.
4. **Special Characters are not Properly Uploaded** The model technically has the international value allowed, however I could not get them to work, and had so many error, and I'm already behind so I gave up on them for this DB and used plain English Alphabet to write any letters that had an accent of some kind.

Lessons Learned

The most instructive moment in building this model came during the design of the transaction structure. The initial impulse was to create entirely separate Purchase Order and Sales Receipt tables with no shared parent. Only when writing the aggregate revenue and expense queries did it become clear that a unified transaction supertype would dramatically simplify cross-cutting financial reports. The `TXN_Transaction_14` supertype was added mid-design, and restructuring the foreign keys around it required revisiting several already-built relationships.

A similar issue to the TXN table was all of the alcohol categories. The intent was to illustrate the 3 specialization types, but when it came to the data entry stage, I realized that it would be best to append regardless of major category in order to ensure unique Identities.

Lastly, I had the biggest struggle with MySQL Workbench, especially with flyig and needing to use my laptop. The Key assignments were causing bugs even though my table feature would be correct if it was and adjustment from a previous name for that entity it wouldn't automatically correct, this resulted in a lot of errors and struggles. But we got there.

Report Queries & Descriptions

Q1 · Self-Join — Brand Siblings by Shared Producer

Description: Pairs of brands in the Brand lookup table that share the same parent producer, ordered by producer and brand name. The condition `b1.brand_id < b2.brand_id` prevents self-matches and eliminates mirror duplicate pairs, so each unique pairing appears exactly once. This reveals which parent companies supply multiple distinct labels carried by the bar — useful when renegotiating vendor contracts or analyzing supplier concentration risk.

```
SELECT *
FROM (
    SELECT
        b1.producer          AS shared_producer,
        b1.producer_category AS category,
        b1.Domestic_Foreign AS origin,
        b1.brand_name       AS brand_A,
        b2.brand_name       AS brand_B
    FROM Brand_LUT_1 b1
    JOIN Brand_LUT_1 b2
        ON b1.producer = b2.producer
        AND b1.brand_id < b2.brand_id
    ORDER BY b1.producer, b1.brand_name
) AS Query1_ProducerMakesForMoreThan1Brand;
-- Returns: 20 rows
```

Table 1: Q1 Result — Brand Siblings by Shared Producer (20 rows)

shared_producer	category	origin	brand_A	brand_B
Buffalo Trace Distillery (Sazerac Company)	Distillery	Domestic	Blantons	William Larue Weller
Buffalo Trace Distillery (Sazerac Company)	Distillery	Domestic	Buffalo Trace	Eagle Rare
Buffalo Trace Distillery (Sazerac Company)	Distillery	Domestic	Buffalo Trace	Blantons
Buffalo Trace Distillery (Sazerac Company)	Distillery	Domestic	Buffalo Trace	William Larue Weller
Buffalo Trace Distillery (Sazerac Company)	Distillery	Domestic	Eagle Rare	Blantons
Buffalo Trace Distillery (Sazerac Company)	Distillery	Domestic	Eagle Rare	William Larue Weller
Constellation Brands Inc.	Winery	Foreign	Kim Crawford	Meiomi Wines
Davide Campari-Milano N.V.	Distillery	Foreign	Campari	Aperol
Freixenet-Mionetto USA Inc.	Winery	Foreign	Freixenet	Gloria Ferrer
Irish Distillers Ltd. (Pernod Ricard)	Distillery	Foreign	Jameson	Redbreast
J. Wray & Nephew Ltd.	Distillery	Foreign	Appleton Estate	Wray and Nephew
Maison Ferrand S.A.S.	Distillery	Foreign	Plantation Rum	Banks Rum
Suntory Spirits Ltd. (Beam Suntory)	Distillery	Foreign	Midori	Haku Vodka
Suntory Spirits Ltd. (Beam Suntory)	Distillery	Foreign	Roku Gin	Midori
Suntory Spirits Ltd. (Beam Suntory)	Distillery	Foreign	Roku Gin	Haku Vodka
Suntory Spirits Ltd. (Beam Suntory)	Distillery	Foreign	Suntory Toki	Roku Gin

Suntory Spirits Ltd. (Beam Suntory)	Distillery	Foreign	Suntory Toki	Midori
Suntory Spirits Ltd. (Beam Suntory)	Distillery	Foreign	Suntory Toki	Haku Vodka
Treasury Wine Estates Ltd.	Winery	Foreign	La Marca Prosecco	Beringer Vineyards
William Grant & Sons Ltd.	Distillery	Foreign	Glenfiddich	Hendricks Gin

Q2 · Multi-Table Join (5 Tables) — Full Tequila Inventory Profile

Description: A complete tequila product profile joining the base product catalog, the liquor supertype, the tequila subtype, and inventory volumes — sorted by age category then product name. `LEFT JOIN` on inventory keeps tequilas not yet assigned stock records. `NULLIF(TRIM(...), '')` surfaces a clean `NULL` rather than a misleading empty string for additive profile fields. This report gives management a full picture of every tequila’s specifications and current availability in a single view.

```
SELECT *
FROM (
  SELECT
    p.product_id,
    p.product_name,
    p.product_brand,
    l8.abv_pct,
    l8.bottle_size_mL,
    t9.age_category,
    t9.cooking_method,
    t9.is_additive_free,
    NULLIF(TRIM(t9.additive_profile), '') AS additive_profile,
    t9.nom_number,
    COALESCE(iv.is_instock, 0) AS is_instock,
    COALESCE(iv.backstock_quantity, 0) AS backstock_qty,
    COALESCE(iv.volume_BOH_stock, 0) AS vol_BOH,
    COALESCE(iv.is_discontinued, 0) AS is_discontinued
  FROM      inventory_product_start_6 p
  JOIN      LIQUOR_8                  l8 ON p.product_id = l8.product_id
  JOIN      LIQsub_Tequila_9          t9 ON l8.product_id = t9.product_id
  LEFT JOIN inventory_volumes_7       iv ON p.product_id = iv.product_id
  WHERE     l8.subtype_LUT = 'Tequila'
  ORDER BY t9.age_category, p.product_name
) AS Query2_Full_Tequila_Product_Info;
-- Returns: 14 rows
```

Table 2: Q2 Result — Full Tequila Inventory Profile

product_id	product_name	product_brand	abv_pct	bottle_size_mL	age_category	cooking_method	is_add
300065	Casa Noble Anejo	Casa Noble	40	750	Anejo	Brick Oven/Horno	
300067	Casamigos Cristalino Anejo	Casamigos	40	750	Anejo	Autoclave	
300071	Don Julio 1942	Don Julio	38	750	Anejo	Autoclave	
300069	El Tesoro Anejo	El Tesoro	40	750	Anejo	Brick Oven/Horno	
300068	Espolon Blanco	Espolon	40	750	Blanco	Autoclave	
300070	Fortaleza Blanco	Fortaleza	40	750	Blanco	Brick Oven/Horno	
300075	Mijenta Blanco	Mijenta	40	750	Blanco	Brick Oven/Horno	
300074	Olmecca Altos Plata	Olmecca Altos	40	750	Blanco	Roller Mill	
300063	Patron Silver	Patron	40	750	Blanco	Autoclave	
300076	Del Maguey Vida Mezcal	Del Maguey	42	750	Blanco/Joven	Pit Roast	
300066	Clase Azul Ultra	Clase Azul	40	750	Extra Anejo	Brick Oven/Horno	
300073	Codigo 1530 Rosa	Codigo 1530	40	750	Reposado	Autoclave	
300064	Don Julio Reposado	Don Julio	38	750	Reposado	Autoclave	

Q3 · Money Calculation View — Bartender Revenue & Tip Percentage

Description: Per-employee summary of sales activity including number of receipts handled, pre-tax sales total, tax collected, tips earned, total tendered, average ticket size, and tip percentage of pre-tax sales. **LEFT JOIN** ensures all employees appear even if they have no associated receipts. **NULLIF** on the denominator guards against divide-by-zero for employees with zero sales. Results are sorted by total tips descending to quickly surface the highest-earning staff.

```
SELECT *
FROM (
    SELECT
        e.employee_id,
        CONCAT(e.first_name, ' ', e.last_name) AS employee_name,
        e.title,
        e.hourly_rate,
        COUNT(sr.transaction_id) AS num_receipts,
        SUM(sr.retail_price) AS total_pretax_sales,
        SUM(sr.tax_amount) AS total_tax_collected,
        SUM(sr.tip) AS total_tips,
        SUM(sr.tendered_amount) AS total_tendered,
        ROUND(AVG(sr.retail_price), 2) AS avg_ticket_size,
        ROUND(
            SUM(sr.tip) / NULLIF(SUM(sr.retail_price), 0) * 100, 2) AS tip_pct_of_sales
    FROM Employees_LUT_5 AS e
    LEFT JOIN TXNsub_SR_17 AS sr ON e.employee_id = sr.employee_id
    GROUP BY
        e.employee_id, e.first_name, e.last_name,
        e.title, e.hourly_rate
    ORDER BY total_tips DESC
) AS Query3_Bartender_TipPercent;
-- Returns: 4 rows
```

Table 3: Q3 Result — Bartender Revenue & T
(4 rows)

employee_id	employee_name	title	hourly_rate	num_receipts	total_pretax_sales	total_tax
1002	Jade Okafor	Bartender	18.50	2	146	
1003	Caitlin Nguyen	Bartender	18.50	2	93	
1001	Marcus Rivera	Bartender	18.50	1	57	
1004	Derek Sullivan	Lead bartender	26.75	1	23	

Q4 · Volume Calculation View — Total Ingredient Pour Volume Across All Sales

Description: Total fluid ounces and milliliters poured per ingredient across all sales receipts, broken out by ingredient type and major product category. A **CASE** statement on the **qualifier** column selects the correct recipe measure (single, double, or happy-hour single) for each line item, multiplied by quantity sold. Garnish and Non-Alcoholic ingredient types are excluded so results reflect only pourable alcohol volumes. Sorted descending by total ounces poured to highlight the highest-consumption products — directly informing reorder priority.

```

SELECT *
FROM (
  SELECT
    r.ingredient_name,
    r.subtype
    AS ingredient_type,
    p.major_category,
    SUM(
      CASE sl.qualifier
        WHEN 'single' THEN r.single * sl.quantity
        WHEN 'double' THEN r.double * sl.quantity
        WHEN 'HH' THEN r.single * sl.quantity
        ELSE 0
      END
    )
    AS total_oz_poured,
    ROUND(SUM(
      CASE sl.qualifier
        WHEN 'single' THEN r.single * sl.quantity
        WHEN 'double' THEN r.double * sl.quantity
        WHEN 'HH' THEN r.single * sl.quantity
        ELSE 0
      END
    ) * 29.5735, 2)
    AS total_mL_poured
  FROM TXN_SR_line_item_18 sl
  JOIN MENUsub_Cocktail_LineRecipe_13 r ON sl.MENU_Cocktail_id = r.cocktail_id
  JOIN inventory_product_start_6 p ON r.product_id = p.product_id
  WHERE r.volume_units = 'oz'
  AND r.subtype NOT IN ('Garnish', 'Non-Alcoholic')
  GROUP BY r.ingredient_name, r.subtype, p.major_category
  ORDER BY total_oz_poured DESC
) AS Query4_TotalSales_ProductID_volumes;
-- Returns: 11 rows

```

Table 4: Q4 Result — Total Ingredient Pour Volume Across All Sales (11 rows)

ingredient_name	ingredient_type	major_category	total_oz_poured	total_mL_poured
Guinness Draught	Other	Brew	40.0	1182.94
Lytton Springs Zinfandel	Other	Wine	20.0	591.47
La Marca Prosecco DOC	Other	Wine	15.0	443.60
Cointreau Triple Sec	Liqueur	Liquor	8.0	236.59
Aperol	Liqueur	Liquor	7.5	221.80
Buffalo Trace Bourbon	Whiskey	Liquor	7.5	221.80
Tito's Handmade Vodka	Vodka	Liquor	5.5	162.65
Tanqueray London Dry Gin	Gin	Liquor	5.5	162.65
Bacardi Superior Rum	Rum	Liquor	5.5	162.65
Patron Silver Tequila	Tequila	Liquor	5.5	162.65
Brandy	Other	Liquor	2.5	73.93

Q5 · Multi-Join + Conditional Aggregation — Cocktail Build Profile

Description: A full build profile for each cocktail including total ingredient count, total recipe volume for single and double pours (oz), and counts of spirit, garnish, and non-alcoholic components per cocktail. Conditional aggregation via `CASE` inside `COUNT` and `SUM` breaks down ingredients by subtype without requiring additional joins or self-joins. Results are ordered by spirit count then total ingredient count descending — useful for training, menu costing, and build-time estimation.

```
SELECT *
FROM (
    SELECT
        c.cocktail_id,
        c.cocktail_name,
        c.single AS sell_single,
        c.double AS sell_double,
        c.HH AS sell_HH,
        c.preparation,
        COUNT(r.line_id) AS total_ingredients,
        SUM(CASE WHEN r.volume_units = 'oz' THEN r.single ELSE 0 END) AS build_oz_single,
        SUM(CASE WHEN r.volume_units = 'oz' THEN r.double ELSE 0 END) AS build_oz_double,
        COUNT(CASE WHEN r.subtype = 'Garnish' THEN 1 END) AS garnish_count,
        COUNT(CASE WHEN r.subtype = 'Non-Alcoholic' THEN 1 END) AS na_ingredient_count,
        COUNT(CASE WHEN r.subtype NOT IN
            ('Garnish', 'Non-Alcoholic', 'Other') THEN 1 END) AS spirit_count,
        c.flavor_profile
    FROM MENU_Cocktail_start_12 c
    JOIN MENUsub_Cocktail_LineRecipe_13 r ON c.cocktail_id = r.cocktail_id
    GROUP BY
        c.cocktail_id, c.cocktail_name, c.single, c.double,
        c.HH, c.preparation, c.flavor_profile
    ORDER BY spirit_count DESC, total_ingredients DESC
) AS Query5_CocktailCategoryVol;
-- Returns: 5 rows
```

Table 5: Q5 Result — Cocktail Build

cocktail_id	cocktail_name	sell_single	sell_double	sell_HH	preparation	total_ingredients	build_oz_single	build_oz_double
77004	Long Island Iced Tea	15	20.00	NULL	Shaken	9	4.75	
77002	Red Sangria	12	NULL	9.00	Built	6	6.50	
77003	Prosecco Spritz	13	NULL	10.00	Built	4	5.50	
77005	Buffalo Trace Bourbon Shot	9	11.00	NULL	Neat	2	3.50	
77001	Dark Velvet Shandy	10	NULL	7.50	Built	3	12.00	

Q6 · Multi-Join Purchase Analysis — PO Line Items with Landed Cost

Description: All Purchase Order line items with per-line tax estimate and fully landed unit cost (unit price $\times (1 + \text{tax rate})$), chaining from the transaction header through the PO subtype to line items and product catalog. Sorted by transaction date descending then line total descending. This report supports accounts payable review, landed-cost analysis for menu pricing decisions, and identification of outstanding (unpaid) invoices via the `is_paid_in_full` flag.

```
SELECT *
FROM (
    SELECT
        t.transaction_date,
        po.invoice_number,
```

```

    po.received_date,
    po.is_paid_in_full,
    li.category_type,
    p.major_category,
    li.product_brand,
    p.product_name,
    li.unit_volume,
    li.unit_name,
    li.quantity,
    li.unit_price,
    li.line_total
    AS subtotal_line,
ROUND(li.line_total * po.tax_pct,      2)  AS line_tax,
ROUND(li.line_total * (1 + po.tax_pct),  2)  AS line_total_with_tax,
ROUND(li.unit_price * (1 + po.tax_pct),   4)  AS landed_cost_per_unit
FROM TXN_Transaction_14      t
JOIN TXNsub_PO_15            po ON t.transaction_id = po.transaction_id
JOIN TXN_PO_line_item_16     li ON po.transaction_id = li.transaction_id
JOIN inventory_product_start_6 p ON li.product_id   = p.product_id
WHERE t.subtype = 'Purchase Order'
ORDER BY t.transaction_date DESC, li.line_total DESC
) AS Query6_PO_totalCostbyLine;
-- Returns: 21 rows

```

Table 6: Q6 Result — P

transaction_date	invoice_number	received_date	is_paid_in_full	category_type	major_category	product_brand	product_name
2026-04-20	INV-2026-006	2026-04-22	1	Brew	Brew	Guinness	Guinness
2026-04-20	INV-2026-006	2026-04-22	1	Brew	Brew	Schilling	Local
2026-04-20	INV-2026-006	2026-04-22	1	Brew	Brew	Sierra Nevada	Hazy
2026-04-20	INV-2026-006	2026-04-22	1	Brew	Brew	Sierra Nevada	Nar
2026-04-10	INV-2026-005	2026-04-12	1	Wine	Wine	Sonoma-Cutrer	Rus
2026-04-10	INV-2026-005	2026-04-12	1	Wine	Wine	Kim Crawford	Mar
2026-04-01	INV-2026-004	2026-04-03	0	Gin	Liquor	Tanqueray	Tan
2026-04-01	INV-2026-004	2026-04-03	0	Rum	Liquor	Diplomatico	Dip
2026-04-01	INV-2026-004	2026-04-03	0	Gin	Liquor	Hendricks	Hen
2026-04-01	INV-2026-004	2026-04-03	0	Rum	Liquor	Appleton Estate	App
2026-04-01	INV-2026-004	2026-04-03	0	Rum	Liquor	Bacardi	Bac
2026-03-20	INV-2026-003	2026-03-22	1	Tequila	Liquor	Patron	Pat
2026-03-20	INV-2026-003	2026-03-22	1	Tequila	Liquor	Don Julio	Don
2026-03-20	INV-2026-003	2026-03-22	1	Tequila	Liquor	Espolon	Esp
2026-03-13	INV-2026-001	2026-03-15	1	Whiskey	Liquor	Buffalo Trace Distillery	Buff
2026-03-13	INV-2026-001	2026-03-15	1	Vodka	Liquor	Titos Handmade Vodka	Tit
2026-03-13	INV-2026-001	2026-03-15	1	Gin	Liquor	Tanqueray	Tan
2026-03-13	INV-2026-001	2026-03-15	1	Rum	Liquor	Bacardi	Bac
2026-01-20	INV-2026-002	2026-02-22	0	Wine	Wine	Ridge Vineyards	Lyt
2026-01-20	INV-2026-002	2026-02-22	0	Wine	Wine	La Marca	Pro
2026-01-20	INV-2026-002	2026-02-22	0	Brew	Brew	Guinness	Gui

Q7 · Window Functions + Derived Table — Cocktail Revenue Ranking

Description: Each cocktail/qualifier combination ranked by total revenue, showing each combination's percentage share of total bar revenue and a running cumulative revenue total. A derived table first aggregates sales by cocktail name and qualifier (**GROUP BY**), then window functions — **RANK()** **OVER** and **SUM()** **OVER** — are applied to the aggregated result. This two-step pattern is required because window functions cannot be nested directly inside **GROUP BY** clauses. The result identifies top revenue drivers and shows how quickly

cumulative revenue is reached across the menu, useful for pricing strategy and happy-hour planning.

```
SELECT *
FROM (
    SELECT
        cocktail_name,
        qualifier,
        times_ordered,
        total_qty_sold,
        total_revenue,
        ROUND(total_revenue / total_qty_sold, 2) AS avg_revenue_per_unit,
        RANK() OVER (ORDER BY total_revenue DESC) AS revenue_rank,
        ROUND(
            total_revenue / SUM(total_revenue) OVER () * 100, 2) AS pct_of_total_revenue,
        SUM(total_revenue) OVER (
            ORDER BY total_revenue DESC
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
        ) AS running_revenue_total
    FROM (
        SELECT
            Cocktail_name AS cocktail_name,
            qualifier,
            COUNT(*) AS times_ordered,
            SUM(quantity) AS total_qty_sold,
            SUM(line_total) AS total_revenue
        FROM TXN_SR_line_item_18
        GROUP BY Cocktail_name, qualifier
    ) ranked_sales
    ORDER BY revenue_rank
) AS Query7_rankCocktailRev;
-- Returns: 9 rows
```

Table 7: Q7 Result — Cocktail Revenue Ranking with Total (9 rows)

cocktail_name	qualifier	times_ordered	total_qty_sold	total_revenue	avg_revenue_per_unit
Long Island Iced Tea	double	2	4	80	
Dark Velvet Shandy	single	4	5	50	
Red Sangria	single	4	4	48	
Long Island Iced Tea	single	2	3	45	
Prosecco Spritz	single	2	3	39	
Buffalo Trace Bourbon Shot	single	2	3	27	
Prosecco Spritz	HH	2	2	20	
Buffalo Trace Bourbon Shot	double	1	1	11	
Red Sangria	HH	1	1	9	

We made it to the end of the document and the semester. Yay. I honestly have learned a lot but that did come with some struggles. I am happy to have taken this class. Thank you for your help in clarifying specializations and self-joins.

Best regards, Yvonne Larkin